# Especificação SDD TaskAnalyzer

**Bootcamp III (CEUB EAD) - Entrega Inicial (Fase 1) Spec-Driven Development e `Governança de Contexto de IA**

---

# 1. Visão Geral e Contrato de Negócio (SDD)

## 1.1 Identificação

| **Campo** | **Valor** |
|---|---|
| Nome completo | Gabriel Marques da Rocha |
| Curso | Ciência da Computação |
| Polo / Turma | Turma A - 0726 |
| E-mail institucional | gabriel.mrocha@sempreceub.com |

## 1.2 Propósito do Módulo TaskAnalyzer

O **TaskAnalyzer** é um módulo de análise de tarefas e produtividade. Ele recebe um conjunto de tarefas registradas (com datas, prioridade e status) e devolve métricas objetivas que apoiam a tomada de decisão e a melhoria contínua do fluxo de trabalho.

Objetivos do módulo:

- Analisar tarefas executadas e gerar métricas de produtividade.
- Calcular o **tempo médio de conclusão** das tarefas, geral e por prioridade.
- Identificar a **taxa de atraso** (percentual de tarefas concluídas após o prazo), geral e por prioridade.
- Gerar **indicadores agrupados por prioridade** (baixa, média, alta).
- Fornecer métricas confiáveis para tomada de decisão e melhoria contínua.

Este documento é o **contrato executável** do módulo: na Fase 2, todo código gerado por assistentes de IA deverá seguir estritamente as entradas, saídas, restrições e cenários aqui definidos. A especificação é a fonte da verdade; a IA é a executora; o desenvolvedor é o arquiteto e o homologador.

## 1.3 Contrato Executável de Interface

### 1.3.1 Entradas (Inputs)

A entrada do analisador é uma **lista de tarefas**. Cada tarefa é um registro com os campos abaixo.

| **Campo** | **Tipo** | **Obrigatório** | **Descrição / Restrições** |
|---|---|---|---|
| id_tarefa | int | Sim | Identificador único da tarefa (inteiro > 0, sem repetição na lista). |
| titulo | str | Sim | Título da tarefa (texto não vazio). |
| prioridade | str | Sim | Valores aceitos: "baixa", "media", "alta". |
| status | str | Sim | Valores aceitos: "concluida", "pendente", "cancelada". |
| data_criacao | datetime | Sim | Data/hora de criação da tarefa (UTC). |
| data_inicio | datetime | Não* | Data/hora de início da execução (UTC). *Obrigatória quando status = "concluida". |
| data_conclusao | datetime | Não* | Data/hora de conclusão da execução (UTC). *Obrigatória quando status = "concluida". |

| Campo | Tipo | Obrigatório | Descrição / Restrições |
|---|---|---|---|
| prazo | datetime | Sim | Data/hora limite para conclusão da tarefa (UTC). |

**Assinatura pública do módulo (não pode ser alterada sem autorização):**

```
def analisar_tarefas(tarefas: list[dict]) -> dict:

    ...
```

### 1.3.2 Saídas (Outputs)

O analisador retorna um dicionário de métricas com os campos abaixo.

| Campo | Tipo | Descrição |
|---|---|---|
| tempo_medio_conclusao_min | float | Tempo médio de conclusão em minutos, considerando as tarefas concluídas (geral). |
| taxa_atraso_percentual | float | Percentual de tarefas concluídas após o prazo, sobre o total de concluídas (geral). |
| quantidade_tarefas | int | Total de tarefas concluídas consideradas no cálculo. |
| indicadores_por_prioridade | dict | Métricas agrupadas por prioridade ("baixa", "media", "alta"): tempo médio, taxa de atraso e quantidade de cada grupo. |

### 1.3.3 Regras de Negócio e Restrições

1. **Somente tarefas com status "concluida"** entram no cálculo de tempo médio e taxa de atraso. Tarefas "pendente" e "cancelada" são desconsideradas das métricas de tempo.
2. **Tempo de conclusão** = data_conclusao - data_inicio, expresso **em minutos**.
3. **Tarefa em atraso** = tarefa concluída cuja data_conclusao é posterior ao prazo.

4. **Datas inconsistentes disparam exceção específica**: tarefa com data_conclusao anterior a data_inicio, ou tarefa concluída sem data_inicio/data_conclusao, dispara `TarefaInvalidaError` com mensagem clara indicando o id da tarefa e o problema.
    - **Nota de decisão**: o enunciado admite duas leituras para dado inconsistente (ignorar a tarefa ou tratar como erro). Esta especificação opta por **falhar de forma explícita** via `TarefaInvalidaError`, em coerência com o Cenário 2 de aceite, porque descartar silenciosamente uma tarefa mascararia um problema de qualidade de dado na origem e produziria métricas sem que o usuário soubesse que uma tarefa ficou de fora do cálculo.
5. **Divisão por zero é tratada com exceção específica**: se não houver nenhuma tarefa concluída válida na entrada (denominador zero para as médias), o analisador dispara `MetricasIndisponiveisError` com mensagem clara e objetiva ao usuário.
6. **Fuso horário**: todas as datas são tratadas em UTC.
7. **Arredondamento**: tempos e percentuais são arredondados com 2 casas decimais.
8. **Sem efeitos colaterais**: o analisador não persiste dados em arquivos ou bancos de dados; recebe a lista, calcula e retorna.
9. **Nenhum comportamento fora deste contrato**: casos não especificados aqui não devem ser assumidos pela IA; dúvidas voltam para o arquiteto da especificação.

---

# 2. Especificação de Cenários de Aceite e Test Harness

## 2.1 Cenários de Aceite (Dado - Quando - Então)

### Cenário 1 - Sucesso (métricas corretas)

- **Dado** um conjunto de tarefas válidas, com diferentes prioridades, prazos e status (concluídas e pendentes),
- **Quando** o analisador de tarefas for executado,
- **Então** as métricas retornadas devem estar corretas, incluindo:
    - tempo médio de conclusão por prioridade e geral;
    - taxa de atraso correta por prioridade e geral;
    - quantidade de tarefas correta por prioridade.

**Exemplo de referência:** dadas 4 tarefas concluídas (2 de prioridade alta com tempos de 60 e 120 minutos, 1 média com 30 minutos concluída após o prazo, 1 baixa com 90 minutos) e 1 tarefa pendente, o analisador deve retornar tempo_medio_conclusao_min = 75.0, taxa_atraso_percentual = 25.0, quantidade_tarefas = 4, e os indicadores por prioridade

coerentes com cada grupo: alta com tempo médio 90.0 min, 0.0% de atraso e 2 tarefas; média com 30.0 min, 100.0% de atraso e 1 tarefa; baixa com 90.0 min, 0.0% de atraso e 1 tarefa (a tarefa pendente não entra em nenhum cálculo).

### Cenário 2 - Exceção / Erro (entradas inválidas)

- **Dado** uma entrada com datas inválidas (ex.: conclusão antes do início) ou a ausência de tarefas concluídas (divisão por zero),
- **Quando** o analisador de tarefas for executado,
- **Então** deve disparar uma exceção específica tratada (`TarefaInvalidaError` ou `MetricasIndisponiveisError`), com mensagem clara e objetiva ao usuário, sem retornar métricas parciais ou silenciosamente erradas.

## 2.2 Planejamento do Test Harness

Os cenários de aceite acima são a origem direta dos **testes automatizados (pytest)** que serão implementados na Fase 2, no arquivo `tests/test_harness.py`. O fluxo é: **cenários escritos, depois testes automatizados, depois validação do código gerado por IA**.

Decomposição dos cenários em casos de teste:

| **Cenário de origem** | **Caso de teste (pytest)** | **O que valida** |
|---|---|---|
| Cenário 1 | test_metricas_gerais_com_tarefas_validas | Tempo médio, taxa de atraso e quantidade gerais com o conjunto de referência. |
| Cenário 1 | test_indicadores_por_prioridade | Métricas de cada grupo (baixa, média, alta) calculadas separadamente. |
| Cenário 1 | test_ignora_tarefas_nao_concluidas | Tarefas pendentes e canceladas não afetam as métricas. |
| Cenário 2 | test_excecao_data_conclusao_antes_do_inicio | `TarefaInvalidaError` com mensagem clara indicando a tarefa problemática. |
| Cenário 2 | test_excecao_sem_tarefas_concluidas | `MetricasIndisponiveisError` quando não há denominador para as médias. |

Diretrizes do harness:

- **Testes antes do código**: o harness é escrito a partir desta especificação, antes da geração do código pela IA. Se um teste não consegue ser escrito, o requisito não está claro e a especificação deve ser revisada primeiro.
- **Fixtures de dados**: conjuntos de tarefas de referência (válidas, com atraso, inválidas, lista vazia) definidos como fixtures do pytest, reutilizados entre casos.
- **Execução e validação contínuas**: `pytest tests/ -v` a cada iteração de geração de código. Qualquer falha bloqueia o avanço para a fase seguinte.
- **Critério de aceite do código gerado por IA**: o código só é considerado apto para homologação humana com 100% do harness verde. O harness é a validação objetiva; a revisão humana é a validação crítica final.

---

# 3. Governança de Contexto e Regras para Agentes de IA (CONTEXT_RULES)

As regras desta seção comporão o arquivo `CONTEXT_RULES.md` do repositório, que é a fonte da verdade fornecida aos assistentes de IA em toda interação de geração de código.

## 3.1 Diretrizes Arquiteturais (obrigatórias)

1. **Python 3.11+** como versão mínima obrigatória.
2. **Type hints** (anotações de tipo) em todas as funções e métodos.
3. Seguir o princípio **Single Responsibility (SRP)** e código limpo conforme **PEP 8**.
4. **Documentação formal no padrão Google style docstrings** para módulos, classes, funções e parâmetros.
5. Funções **pequenas, coesas e com um único propósito**.
6. **Tratamento de exceções específico** e com mensagens claras (exceções customizadas do domínio, nunca `except Exception` genérico silencioso).
7. **Logs estruturados** utilizando o módulo padrão `logging` (nunca `print` para diagnóstico).
8. Código **legível, manutenível e com nomes de variáveis e funções descritivos**, em português consistente com o domínio do contrato.

## 3.2 Proibições Explícitas (regras restritivas)

1. **Não** utilizar bibliotecas externas não autorizadas (o módulo usa apenas a biblioteca padrão do Python; pytest é permitido somente no harness de testes).
2. **Não** alterar, remover ou adaptar os **cenários de teste** fornecidos.

3. **Não** modificar a estrutura de pastas definida nesta especificação.
4. **Não** persistir dados em arquivos ou bancos de dados.
5. **Não** alterar a assinatura (nome e parâmetros) das funções públicas sem autorização.
6. **Não** gerar código sem testes correspondentes para as novas funcionalidades.
7. **Não** inserir código duplicado ou sem necessidade.
8. **Não** assumir comportamentos não especificados no contrato de negócio.

## 3.3 Regras de Interação com a IA

1. Fornecer à IA o **contexto completo antes da solicitação**: esta especificação + CONTEXT_RULES.md.
2. **Validar se a IA compreendeu o contrato** antes de gerar código (pedir que ela reformule o que vai fazer).
3. **Solicitar explicações** quando houver dúvida na solução proposta.
4. **Revisar criticamente** todo código gerado antes de executar o harness.
5. **Rejeitar respostas que violem as proibições ou o contrato**, reapresentando a regra violada em vez de aceitar o desvio.

---

# 4. Arquitetura do Repositório e Preparação para Git/GitHub

A arquitetura do repositório da Fase 2 (árvore de arquivos `sdd-task-analyzer/`, papel de cada componente, plano de homologação humana e estratégia de versionamento) está detalhada na **apresentação que acompanha esta entrega** (Google Apresentações), conforme a estrutura de submissão da disciplina.

**Link da apresentação:**
https://docs.google.com/presentation/d/1kmHjIxWOe4P8MkhB7oqJ1tnAnc27NnpgYololry2vJ4/edit?usp=sharing

---

*Documento elaborado como contrato executável da Fase 1 do projeto TaskAnalyzer, no paradigma Spec-Driven Development. Fases seguintes: Fase 2 (implementação com IA e versionamento no GitHub) e Fase 3 (CI/CD, containerização, testes e revisão ética).*